# Scaling Self-Play with Self-Guidance

**Luke Bailey, Kaiyue Wen, Kefan Dong, Tatsunori Hashimoto, Tengyu Ma**

Stanford University

**Abstract**

LLM self-play algorithms are notable in that, in principle, nothing bounds their learning: a Conjecturer model creates problems for a Solver, and both improve together. However, in practice, existing LLM self-play methods *do not scale well with large amounts of compute*, instead hitting learning plateaus. We argue this is because over long training runs, the Conjecturer learns to hack its reward, collapsing to artificially complex problems that do not help the Solver improve. To overcome this, we introduce Self-Guided Self-Play (SGS), a self-play algorithm in which the language model itself guides the Conjecturer away from degeneracy. In SGS, the model takes on three roles: Solver, Conjecturer, and a *Guide* that scores synthetic problems by their relevance to unsolved target problems and how clean and natural they are, providing supervision against Conjecturer collapse. Our core hypothesis is that language models can assess whether a subproblem is useful for achieving a goal. We evaluate the scaling properties of SGS by running training for significantly longer than prior works and by fitting scaling laws to cumulative solve rate curves. Applying SGS to formal theorem proving in Lean4, we find that it surpasses the asymptotic solve rate of our strongest RL baseline in fewer than 80 rounds of self-play and enables a 7B parameter model, after 200 rounds of self-play, to solve more problems than a 671B parameter model pass@4.

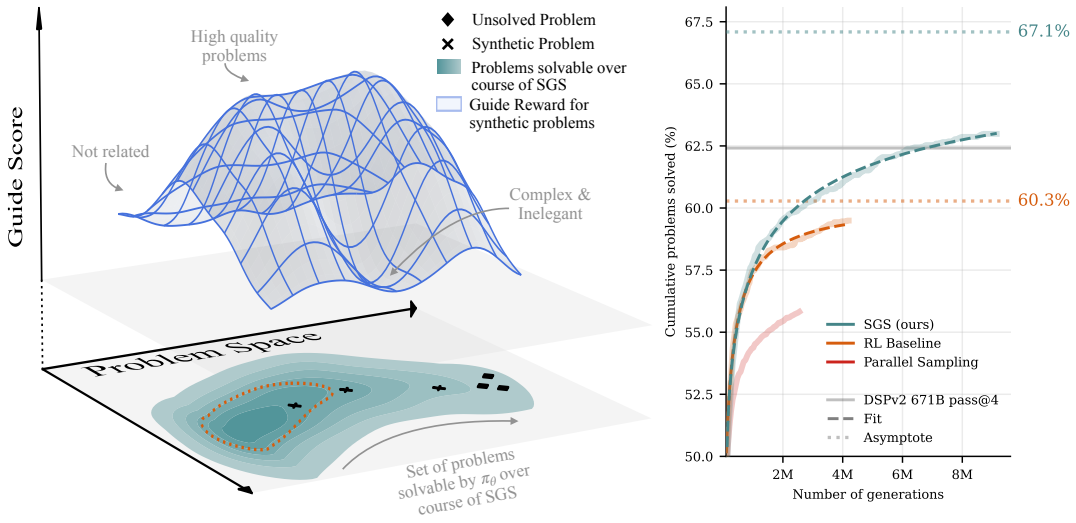


Figure 1: **Left:** Intuition behind SGS. Bottom, the space of problems solvable over course of SGS, starting as a **small area**, expanding past **asymptotic RL performance**, directed towards unsolved target problems (⧫) by synthetic problems (×). Top, the Guide score for synthetic problems, which is high along the path to unsolved targets, and low for not related and inelegant problems (see Figure 2). **Right:** Results of running SGS on $D_{3k}$ (≈ 3000 Lean4 formal math problems) compared to our best performing RL baseline (§4.3). At 6.3M generations, SGS applied to the 7B parameter DeepSeek-Prover-V2 model exceeds the pass@4 of the larger 671B counterpart (DSPv2 671B).

Correspondence to ljbailey@stanford.edu

Code available at https://github.com/LukeBailey181/sgs

# 1 Introduction

We would like AI systems that, given enough compute, can solve any problem we give them. Unfortunately, when a problem is sufficiently difficult, standard reinforcement learning techniques fail due to sparse or absent reward (Prakash & Buvanesh, 2025; Qu et al., 2026). Asymmetric self-play offers a solution to the sparse reward problem. In this framework, a Conjecturer model proposes new learning problems that bridge the gap between the current Solver capability and the unsolved problems (Sukhbaatar et al., 2017; Florensa et al., 2018). Over time, the Solver and Conjecturer improve together.

Self-play algorithms are unique in that, in principle, nothing bounds their learning (Silver et al., 2017). However, existing LLM self-play methods do not sustain learning over long periods (Yu et al., 2025b; Chae et al., 2025). In general, these methods reward the synthetic problems only according to the Solver's pass rate on the problem (Zhao et al.; Huang et al., 2025; Chen et al., 2025c; Liu et al., 2025; Yang et al., 2026; Jana et al., 2026; Kuba et al., 2025). Over time the Conjecturer hacks this reward function, collapsing to artificially complex problems that are not useful for learning (Figure 2) (Dong & Ma, 2025). That is, *the quality of the synthetic problems degrades over the course of training*, leading to a learning plateau.

We can quantify this plateauing problem by fixing a set of hard training problems and seeing how many we can solve with self-play. We strive for an algorithm that, if you run it for long enough, leads to all the problems being solved. In the pursuit of this ideal, we ask the question:

***Given a fixed set of hard problems, how can we scale self-play to solve as many as possible?***

Towards this end, we propose Self-Guided Self-Play (SGS). SGS is an asymmetric self-play algorithm in which an LLM takes on three roles, Solver, Conjecturer, and Guide. It has two mechanisms to stop Conjecturer degradation over the course of training. First, for each unsolved problem, the Conjecturer is prompted to produce a synthetic problem that is useful for solving that problem. Secondly, the Guide is used to score each synthetic problem by how relevant it is to its corresponding unsolved problem, and how clearly it is formulated. Crucially, a synthetic problem that is superficially related to a target but is messy, overly complex, or inelegant, is downweighted by the Guide. The Solver is rewarded for every problem it solves, while the Conjecturer is rewarded for producing problems that the Solver can make progress on and that the Guide judges to be high quality. Our core hypothesis is that language models can assess whether a subproblem is useful for achieving a goal, and thus guide the learning process towards that end.

To evaluate the scaling dynamics of SGS, we take two complementary approaches. First, we run training for much longer than prior works: our main experiment produces over 6 billion tokens during the self-play procedure, epoching the target data over 230 times (Zhao et al.; Huang et al., 2025; Chen et al., 2025c). Second, we fit scaling laws to the cumulative solve rate over training, allowing us to extrapolate long-run behavior and compare the asymptotic solve rates of different methods (Khatri et al., 2025; Ruan et al., 2024).

We use this evaluation method to analyze the long-running dynamics of SGS applied to formal theorem proving in Lean4, where the goal is to generate formally verified proofs of mathematical statements (De Moura et al., 2015; Moura & Ullrich, 2021). In this setting, the target problems are Lean theorems, and solutions are proofs that can be automatically checked by the Lean compiler. We provide a summary of our contributions below:

- In §4.3, we show that SGS outperforms RL and parallel sampling baselines, achieving a 7% higher asymptotic solve rate than RL alone. With sufficient compute, SGS enables DeepSeek-Prover-V2-7B to surpass the pass@4 of the much larger DeepSeek-Prover-V2-671B model (Ren et al., 2025).
- In §4.4, we study the design of the Conjecturer training algorithm. We ablate conditioning the Conjecturer on unsolved problems and the Guide component. In both cases we find the Conjecturer collapses to producing degenerate problems that do not help the Solver make progress on the training problems. We also show that freezing the Conjecturer is suboptimal, as the Solver quickly saturates the fixed problem distribution.
- In §4.5, we study the design of the Solver training algorithm. We show that Solver entropy collapse can starve the Conjecturer of training signal, making the choice of Solver RL objective critical for long-running self-play.

Overall, our key findings are as follows: (1) scaling asymmetric self-play requires avoiding undesirable collapses in the Solver and Conjecturer distributions, which can be achieved by (2) selecting the correct Solver objective, conditioning the Conjecturer on unsolved problems, and adding a data quality reward. We instantiate these ideas in SGS, demonstrating that LLMs themselves can be used as the data quality metric.

## 2 Related Work

**Asymmetric Self-Play.** SGS is an instance of *asymmetric self-play*, methods in which agents with asymmetric roles, typically a Conjecturer generating tasks and Solver solving them, learn through interaction. This paradigm traces its roots to Sukhbaatar et al. (2017) and Florensa et al. (2018), who demonstrate that a proposer can generate goals for a solver in navigation and control environments. Asymmetric self-play hails from earlier work on self-play, involving an agent playing a zero-sum game against itself (Samuel, 1959; Tesauro et al., 1995; Thrun, 1994; Silver et al., 2016).

More recently, the asymmetric self-play paradigm has been applied to tasks with natural language action spaces using LLMs. Some use real data to ground the generated problems (Liu et al., 2025; Yu et al., 2025b; Choi et al.; Sundaram et al., 2026; Jana et al., 2026; Yang et al., 2026), and others consider Conjecturers that use *no seed data* (Huang et al., 2025; Zhao et al.; Xia et al., 2025; Chen et al., 2025c; Kuba et al., 2025; Li et al., 2026). In terms of application domain, Dong & Ma (2025) and Poesia et al. (2024) are most closely related to our work. Both apply asymmetric self-play to formal theorem proving. Dong & Ma (2025) generate formal statements conditioned on *solved problems*. Poesia et al. (2024) consider a setting where an agent builds up mathematics from scratch starting with only axioms.

Our work differs from prior works in three ways. Firstly, the majority of existing algorithms *only reward the Conjecturer according to the Solver performance*, which we find leads to synthetic problem degradation (§4.4, Figure 2). We address this using the Guide model. We only see this degradation issue because secondly, we scale the self-play procedure to longer runs than prior works. We do so because thirdly, we are motivated to study the scaling properties of self-play to solve problems far beyond the base model capability, and thus overcome learning plateaus.

**LLMs for Formal Math.** We use formal math as the application area to test SGS. The current state-of-the-art methods that apply LLMs to formal mathematics use complex inference time strategies and agent scaffolds (Chen et al., 2025b,d; Varambally et al., 2025). In this work, we use the simpler whole proof generation method, where an LLM takes as input the formal problem,

and generates an entire proof directly, possibly after some natural language reasoning (Lin et al., 2025a,b; Ren et al., 2025). There is no reason, however, that SGS could not be used to train a complex agent-based prover, but we leave this to future work.

# 3 The SGS Algorithm

We consider a fixed set of problems $\mathcal{D} = \{x_1, \ldots, x_N\}$ which we aim to solve. We assume each problem admits a verifiable solution; in our work, problems are formal math questions in Lean4. Our goal is to solve as many of the problems in $\mathcal{D}$ as possible given a large compute budget. SGS has three components: (1) a Solver $\pi_\theta$, (2) a Synthetic Data Conjecturer $g_\phi$, and (3) a Guide $\rho$, an LLM-based judge that evaluates synthetic data quality. All of these are *initialized from the same base model*. Each iteration of the algorithm proceeds as follows (see Algorithm 1):

1. **Synthetic problem generation**: Sample a batch $\mathcal{B} \subseteq \mathcal{D}$ of problems. For each problem $x \in B$ not yet solved by the Solver, the Conjecturer produces a related synthetic problem $\tilde{x} \sim g_\phi(\cdot \mid x)$, which is intended to be simpler but meaningfully related. In full generality, the Conjecturer would have to produce an entire Markov Decision Process (MDP), including a reward function, to train the Solver. In our work, the formal problem produced by the Conjecturer comes with a reward function, the Lean4 compiler.

2. **Rollout and verification**: The Solver attempts to solve both the original problems $x \in \mathcal{B}$ and the synthetic problems $\tilde{x}$, and rewards are calculated.

3. **Solver and Conjecturer update**: the Solver is updated using a reinforcement learning objective over rollouts on both original and synthetic problems. The Conjecturer is updated using a reward that favors useful synthetic problems; for each $\tilde{x}$, the reward combines (a) a difficulty signal proportional to the Solver's empirical solve rate on $\tilde{x}$, favoring problems of intermediate difficulty, and (b) a score $\rho(x, \tilde{x}) \in [0, 1]$ produced by the Guide LLM, indicating how well $\tilde{x}$ relates to its target problem $x$, and how elegant a conjecture it is.

## 3.1 Synthetic Data Generation and Verification (Steps 1, 2)

We partition the sampled batch of data $\mathcal{B} = \{x_i\}_{i=1}^{B} \subseteq \mathcal{D}$ into two subsets: $\mathcal{B}_{\text{solved}}$, containing problems with a correct solution from prior rounds, and $\mathcal{B}_{\text{unsolved}}$, containing unsolved problems. For each problem $x \in \mathcal{B}_{\text{unsolved}}$, the Conjecturer $g_\phi$ produces a synthetic problem $\tilde{x} \sim g_\phi(\cdot \mid x)$. The Conjecturer is prompted to create a version of $x$ that is conceptually related but simpler to solve (see Appendix §E for prompt). For problems in $\mathcal{B}_{\text{solved}}$, no synthetic problems are generated. Let $\mathcal{B}_{\text{synth}}$ be the set of synthetic problems generated. For each problem in $\mathcal{B} \cup \mathcal{B}_{\text{synth}}$, the Solver $\pi_\theta$ generates $k$ attempts. For each solution $y_x^i$ for problem $x$, we determine its correctness $v(y_x^i) \in \{0, 1\}$ (for our experiments, using the Lean4 compiler).

## 3.2 Solver and Conjecturer Update (Step 3)

Next we update the parameters of the Solver $\pi_\theta$ and the Conjecturer $g_\phi$ using the collected trajectories. For our experiments, we do not tie the weights of the Solver and Conjecturer and thus we perform separate optimization steps.[0]

[0]Tying weights between Solver, Conjecturer, and Guide is entirely possible, however we do not explore it in this work.

**Solver Update.** The Solver $\pi_\theta$ is updated on correct solutions. For each rollout $y_x^i$ the reward is its binary verification $v(y_x^i)$. In principle, any reinforcement learning objective can be used to update the Solver using $v(y_x^i)$. We use a REINFORCE objective on all problems with solve rate less than or equal to 0.5, promoting learning on hard problems. As the reward is binary, this reduces to a simple log likelihood objective over all correct rollouts for problems with solve rate less than or equal to 0.5 (see §G for more details). We refer to this objective as REINFORCE$^{1/2}$, and justify its use over other RL objectives in §4.3.

**Algorithm 1** SGS

**Require:** Dataset $\mathcal{D}$, Solver $\pi_\theta$, Conjecturer $g_\phi$, Guide $\rho$

Initialize all problems in $\mathcal{D}$ as unsolved
**for** iteration $t = 1, 2, \ldots$ **do**
  Sample batch $\mathcal{B} \subseteq \mathcal{D}$
  Split $\mathcal{B}$ into $\mathcal{B}_{\text{solved}}, \mathcal{B}_{\text{unsolved}}$
  **for** each $x \in \mathcal{B}_{\text{unsolved}}$ **do**
    Generate $\tilde{x} \sim g_\phi(\cdot \mid x)$
  Collect synthetic set $\mathcal{B}_{\text{synth}}$
  **for** each problem $x \in \mathcal{B} \cup \mathcal{B}_{\text{synth}}$ **do**
    Sample $k$ solutions from $\pi_\theta$
    Verify each solution $v(y_x^i) \in \{0, 1\}$
  **for** each $\tilde{x} \in \mathcal{B}_{\text{synth}}$ **do**
    $s(\tilde{x}) \leftarrow$ solve rate of $\tilde{x}$
    $\text{ind} \leftarrow \mathbf{1}[s(\tilde{x}) \neq 0 \wedge s(\tilde{x}) \text{ in bottom } 70\%]$
    $R_{\text{solve}} \leftarrow \text{ind} \cdot (1 - s(\tilde{x}))$
    $R_{\text{guide}} \leftarrow \rho(x, \tilde{x})$
  Update Solver $\pi_\theta$ with RL using $v(y_x^i)$
  Update Conjecturer $g_\phi$ with RL using $R_{\text{solve}} \cdot R_{\text{guide}}$

**Conjecturer Update.** The goal of the Conjecturer is to produce synthetic problems $\tilde{x}$ that are both relevant to the conditioned target $x$ and neither too simple nor impossible. We define the reward for a synthetic problem as $R_{\text{synth}} = R_{\text{solve}} \cdot R_{\text{guide}}$. To update the conjecturer with $R_{\text{synth}}$ we linearly normalize it within its batch to [0,1] (see §G.5).

*Solve Rate Reward ($R_{solve}$):* Let $s(\tilde{x}) = \frac{1}{k}\sum_{i=1}^{k} v(y_{\tilde{x}}^i)$ be the solve rate of the synthetic problem. We set $R_{\text{solve}} = 0$ if $s(\tilde{x}) = 0$ (too difficult) or if $s(\tilde{x})$ is in the top 30% of solve rates within the current batch (too easy). For all other problems, we set $R_{\text{solve}} = 1 - s(\tilde{x})$, thereby favoring harder problems within the solvable range.

*Guide Reward ($R_{guide}$):* We use a reviewer model $\rho$ that is the same model as the initial Solver and Conjecturer. The reviewer is prompted to evaluate the quality of $\tilde{x} \sim g_\phi(\cdot|x)$ relative to the unsolved $x$, outputting a score $R_{\text{guide}} = \rho(x, \tilde{x})$. The details of this prompt greatly affect the synthetic data. For our experiments, we ran a long testing run, iteratively viewing generated problems, identifying flaws in them, and prompting the reviewer to down-weight such problems. In the end, we had a fairly simple rubric that outputted high scores for problems that were (1) related to the unsolved problem, and (2) were formulated clearly by having a simple conclusion and no redundant premises. We provide further details, and the reviewer prompts used, in §E. Like the Solver, we use a simple REINFORCE objective to update the Conjecturer. We provide details in Appendix §G.

# 4 Experimental Results

## 4.1 Implementation Details

**Dataset and training details.** For our main experiments, we use a subset of the Goedel-Pset-V1 (Lin et al., 2025a), a dataset of auto formalized problems primarily from the NuminaMath-CoT dataset (Li et al., 2024) (itself a collection of math problems spanning from grade school to undergraduate level). Our experiments are focused on trying to solve a small fixed set of problems over a long

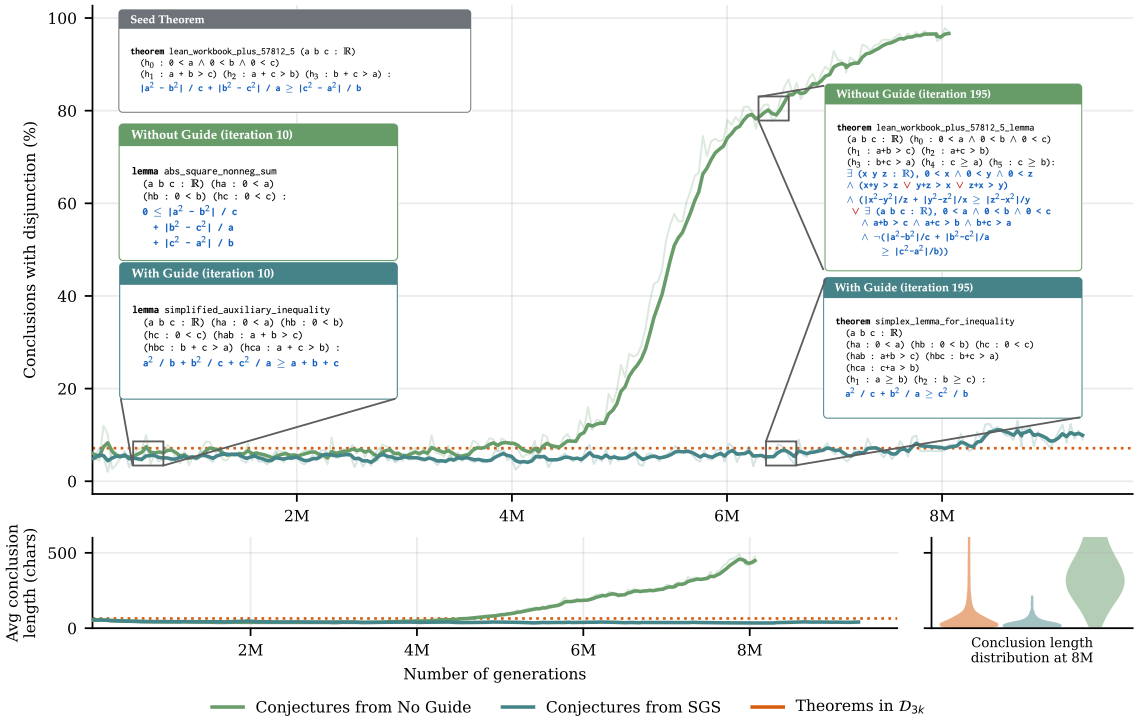


Figure 2: **Benefit of Guide component in SGS**. **Top:** Percentage of generated problems with disjunctive conclusions. Without the Guide, this rises to over 80%; SGS stays near the $D_{3k}$ baseline (**dotted orange**). **Bottom:** Average conclusion length, confirming that No Guide problems become overly long and complex. **Inlaid examples:** Problems generated from the same unsolved seed theorem, problem conclusions are shown in **blue**. By iteration 195, SGS generates a related problem that assumes an ordering of side lengths (such an assumption is required to solve the seed problem with cases) while No Guide produces a convoluted statement with disjunctions ($\vee$).

training run. Accordingly, we sample 5,000 problems from Goedel-Pset-V1. Auto formalization methods can result in problems that are impossible to prove due to a formalization error. Because of this we remove any problems that GPT 5 mini determines are impossible to prove due to a formalization error. This results in our main experimental dataset of 3,323 problems, hereafter referred to as $D_{3k}$ (for more details see Appendix §D). For all experiments, we set our batch size to the total number of problems (thus we sample rollouts for all 3,323 problems), and do 8 proof rollouts per problem. We use the full batch setting because we get high hardware utilization by running a longer rollout phase, and we remove any batch variance from metrics. We use custom training infrastructure to utilize a heterogeneous GPU and CPU cluster, detailed in Appendix §B.

We apply a length penalty to all RL updates inspired by Soft Overlong Punishment (Yu et al., 2025a). Rollouts that consume between 80% and 100% of the context window receive a negative reward that scales linearly from 0 to $-1$. We also give 0 reward to any Solver proof with the `try` tactic, as we found it common for the Solver to get caught in infinite loops using this tactic.

**Model.** We use DeepSeek-Prover-V2-7B (Ren et al., 2025), a model trained with large-scale RL for Lean4, to initialize the Solver, Conjecturer, and Guide in SGS. Because of the Lean-specific RL, the

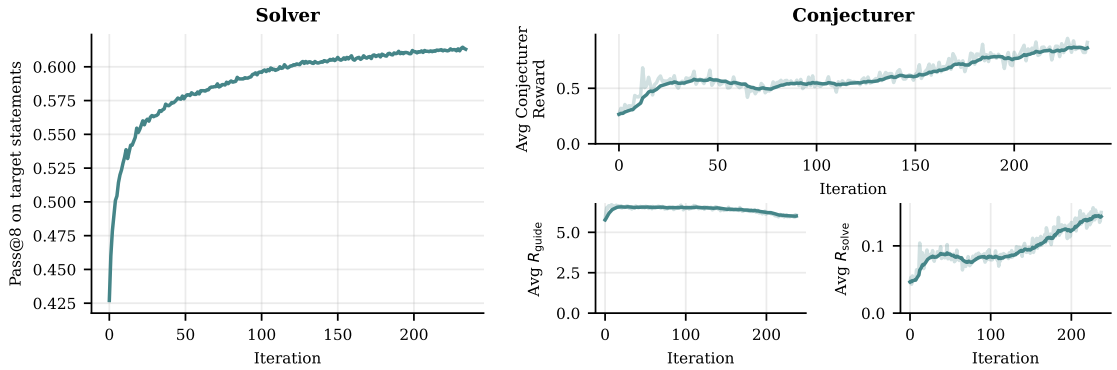


Figure 3: Training dynamics of SGS. **Left:** Solver pass@8 on target statements increases steadily. **Right:** Conjecturer metrics, average Conjecturer reward (top, varies between 0 and 7), average Guide reward (bottom left, varies between 0 and 8), and average solve rate reward (bottom right, varies between 0 and 7/8). We see $R_{\text{guide}}$ begins high (due to initial Conjecturer prompt providing a good prior) and *critically remains there*, while $R_{\text{solve}}$ increases during training.

model has poor instruction following capabilities. In the Conjecturer role, at the start of SGS it outputs the correct format to extract a synthetic problem 51.5% of the time, which is acceptable as the Conjecturer RL increases this. For the Guide, the base model outputs the correct format 54.7% of the time. As the Guide is not trained, this error rate would persist, and is thus unacceptable. To alleviate this, we run a small SFT stage. We take 2048 examples of well formatted reviewer outputs, created using GPT 4.1 mini, and finetune DeepSeek-Prover-V2-7B on them. This increases the number of well formatted reviewer outputs to over 99%.

## 4.2 Modeling scaling performance

Unlike pre-training scaling laws which model loss, our setting requires modeling a bounded accuracy metric, specifically the cumulative solve rate on training problems. Following other works modeling accuracy metrics (Khatri et al., 2025; Ruan et al., 2024), we adopt a sigmoidal curve (with respect to log compute) of the form

$$R_C = R_0 + (A - R_0) \cdot \frac{1}{1 + (C_{\text{mid}}/C)^B}, \tag{1}$$

where $R_C$ is the cumulative solve rate after $C$ model generations (our proxy for compute), $R_0$ is the initial solve rate, $A$ is the asymptotic cumulative solve rate, $C_{\text{mid}}$ is the curve midpoint, and $B$ controls the rate at which performance approaches the asymptote. We provide details of how we fit the scaling laws in Appendix §A.

**Validation of fits**. Our fits are stable when removing 10%, 20%, and 30% of the final training data, and are robust when randomly dropping 50% of training points: the fitted asymptote varies by 1.1% (Figure 8, Appendix §A). We therefore treat differences below 1.1% cautiously. Since true asymptotic solve rates are unobservable, we use fitted scaling laws as a useful *additional* metric rather than a substitute for long-run empirical performance.

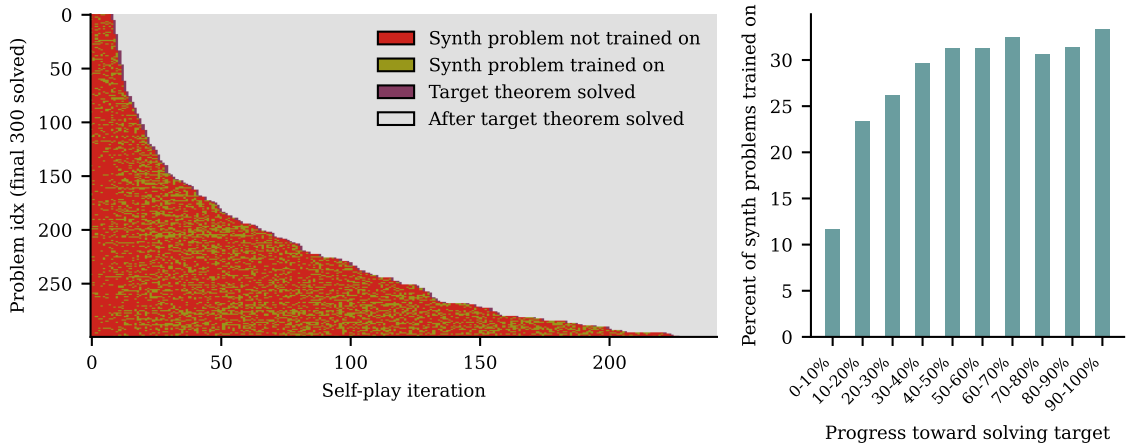


Figure 5: **More synthetic problems are solved as you approach solving a theorem. Left:** A heatmap of the last 300 target theorems to be solved by SGS in Figure 1. Each row shows the training history for a single target theorem. **Green** and **red** cells mark iterations in which the theorem's synthetic problem is and is not suitable for training on respectively. **Right:** For each of the final 300 target theorems, we bin the pre-solve iterations into ten equally-sized buckets and compute the fraction of iterations in that bucket in which the theorem's synthetic problem is trained on. Bars show the average across the final 300 target theorems. We see as the Solver approaches solving each target problem, it is trained on more synthetic problems for said target.

## 4.3 SGS performance

**RL baselines.** We begin by carefully benchmarking three different RL baselines on $D_{3k}$. As a representative example of grouped RL objectives, the dominant paradigm of LLM RL, we use CISPO (Chen et al., 2025a). We select this over other grouped RL objectives as Khatri et al. (2025) demonstrate it has better asymptotic validation accuracy than other methods. We also test a variant of Expert Iteration suggested by Dong & Ma (2025), which involves only sampling solutions for any problem that we have solved fewer than 16 times. Finally, we test REINFORCE$^{1/2}$ as introduced in §3. For each algorithm, we tune the learning rate on $D_{3k}$ to ensure optimal performance. We provide further details in §G. Overall we find REINFORCE$^{1/2}$ is the best performing (Figure 4). CISPO performs poorly due to entropy collapse (Figure 11). In §4.5 we show that this entropy collapse makes vanilla CISPO unsuitable as a Solver update objective for SGS.

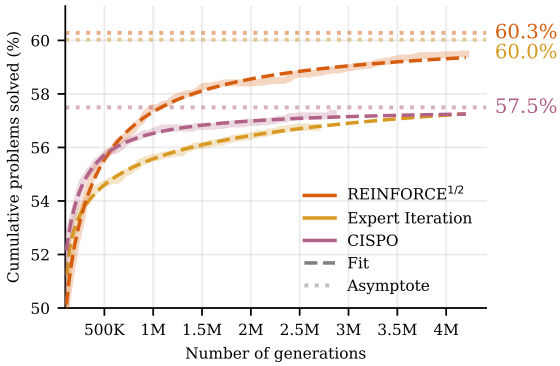


Figure 4: Performance of each RL baseline method on $D_{3k}$. REINFORCE$^{1/2}$ performs best, closely followed by EI. CISPO performs worst due to entropy collapse, shown in Figure 11.

**Main result.** From Figure 4 we select REINFORCE$^{1/2}$ as the best performing RL baseline. We run SGS on $D_{3k}$, and compare against this RL baseline and parallel sampling. The results are shown in Figure 1. Firstly, we see the performance of SGS is consistently better than the RL baseline, with a 7% higher asymptotic cumulative solve rate. If we narrow our view to only the problems that the

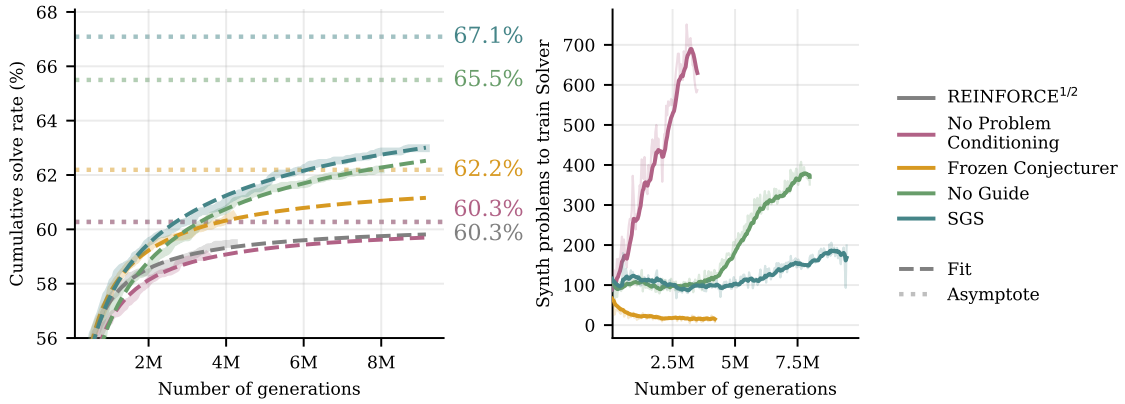


Figure 6: Ablation of the SGS components. **Left:** Cumulative solve rate. The Guide improves performance at all points during training. Freezing the Conjecturer leads to even worse performance, but still above baseline. No Problem Conditioning shows no improvement over baseline. **Right:** Number of solvable problems used to train the Solver per iteration. Without the Guide, the Conjecturer produces far more solvable problems, but this does not improve solve rate. With a frozen Conjecturer, the fixed distribution of synthetic problems is quickly learned.

RL baseline never solves (of which there are 1346), we find SGS sees steady and sustained progress from the very beginning on such problems, solving almost 10% after 8M generations compared to 0% for RL after 4M generations (see Figure 13 of Appendix §F.3).

As a yardstick to compare SGS performance to, we use the pass@4 of the much larger 671B parameter DeepSeek-Prover-V2 model. We see that at 6.3M generations, SGS applied to the 7B parameter DeepSeek-Prover-V2 model exceeds the pass@4 of the larger 671B counterpart. We also compare SGS to STP (Dong & Ma, 2025), the closest related self-play method designed *specifically for Lean4 theorem proving*. We find that SGS has superior scaling, outperforming STP after 1M generations (Figure 12).

In Figure 3 we show the training dynamics of SGS. We see that over training, the Solver and Conjecturer both improve reliably. The Solver pass@8 on the target statements increases steadily, demonstrating that the Solver is not suffering from catastrophic forgetting. For the Conjecturer, we see that the overall reward increases across training, primarily driven by a stable reviewer reward, and increasing solve rate reward. In Figure 5 we show information about what synthetic problems are solved during training. Interestingly, from Figure 5 right panel, we see as the Solver approaches solving a target problem, it is trained on more synthetic problems for said problem. This is what one would expect should the synthetic problems for a specific target be useful for solving that target.

## 4.4 The benefit of self-guidance in the Conjecturer algorithm

We conduct a number of ablations on SGS. Recall that SGS has two components that influence the Conjecturer distribution, (1) conditioning the Conjecturer with unsolved problems, (2) using the Guide to reward the Conjecturer. We ablate both and show the results in Figure 6. All ablations use REINFORCE$^{1/2}$ as the Solver objective and are run on $D_{3k}$.

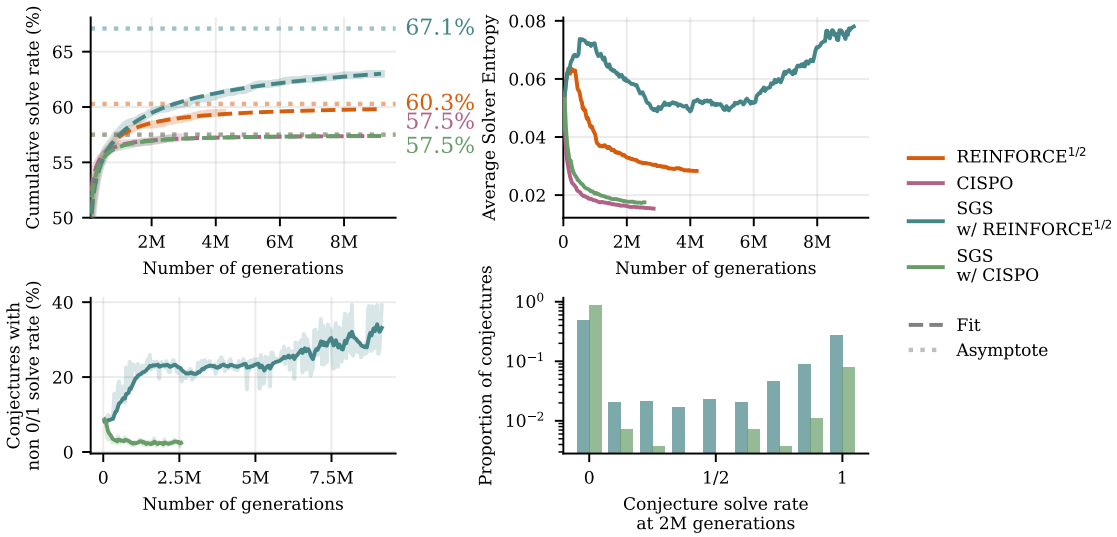


Figure 7: SGS with CISPO vs. REINFORCE$^{1/2}$ as the Solver objective. CISPO's entropy collapse (top right) concentrates problem solve rates at 0 and 1 (bottom right), starving the Conjecturer of reward and preventing it from learning (bottom left). With REINFORCE$^{1/2}$, entropy remains stable and the Conjecturer learns.

**No Problem Conditioning.** This involves running the Conjecturer without conditioning on unsolved problems, and not using the Guide reward (we only reward the Conjecturer using $R_{\text{solve}}$). We simply sample problems from the conjecturer with a fixed prompt (see Appendix §E), however still sample 1 problem per unsolved target. We see that this method *does not outperform the RL baseline*. Indeed while the Conjecturer is able to produce many problems that are difficult for the Solver (Figure 6 right), they are completely useless for solving more target problems.

**No Guide.** We compare the full SGS algorithm against an identical configuration in which we remove $R_{\text{guide}}$ from the Conjecturer reward, so that the Conjecturer is trained only on $R_{\text{solve}}$. We refer to this variant as "No Guide." SGS with the Guide achieves a higher cumulative solve rate than the No Guide variant at every point during training, and the fitted asymptotic solve rate is also higher: 67.1% versus 65.5% (Figure 6, left). Both variants substantially outperform the REINFORCE$^{1/2}$ baseline and No Problem Conditioning.

Similar to No Problem Conditioning, the No Guide variant's lower performance is surprising given that it produces more training data for the Solver (Figure 6, right). Naïvely, more solvable problems should mean more training signal and faster learning. However, once again, *the additional problems do not translate to better solve rates on target problems*. We demonstrate why this is in Figure 2. At 4M generations, when the No Guide Conjecturer begins producing substantially more solvable problems, it collapses to producing problems with long and complex conclusions, often involving many disjunctions (OR) of clauses. In Figure 2 top, we show the number of disjunctive conclusions increasing and saturating to near 100%, far higher than the base rate of less than 10% present in the theorems of $D_{3k}$. In addition, Figure 2 bottom shows the average length of conclusions increasing to nearly 10 times that of $D_{3k}$. In contrast to this, the SGS Conjecturer stably produces problems with similar lengths and number of disjunctive conclusions as $D_{3k}$. The Guide in SGS ensures stability by down-weighting problems that are superficially related to the target but inelegant.

**Frozen Conjecturer.** Our prior ablation demonstrates that the Guide component helps to stabilize the long-running learning of the Conjecturer, avoiding collapsing to problems that hack the solve rate reward. Another much simpler method to stabilize the Conjecturer is to simply not train it. Indeed, this is the test time RL approach used in AlphaProof (Hubert et al., 2025), where a fixed Gemini model is used as the Conjecturer. We see that this method is better than the RL-baseline and No Problem Conditioning ablation, but inferior to both the full SGS method, and the No Guide ablation. Inspecting the Figure 6 right panel, we can see why this may be the case. With a frozen Conjecturer, the number of synthetic problems to train the Solver decreases over time, demonstrating that the Solver capability quickly covers the distribution of synthetic problems from the fixed Conjecturer. When the Solver is starved of synthetic problems, the algorithm reduces to the RL baseline.

### 4.5 Importance of managing entropy in the Solver algorithm

In §4.3 we observed that CISPO produces rapid entropy collapse when used as an RL baseline. Here we show a general connection between the Solver's entropy and Conjecturer's learning objective.

In Figure 7 we show the training dynamics of SGS when using CISPO as the Solver objective. The asymptotic solve rate remains *essentially identical* to standalone CISPO. When CISPO is used as the Solver objective, the Solver's entropy collapses rapidly (Figure 7, top right). As the Solver becomes near-deterministic, the distribution of problem solve rates concentrates at 0 and 1 (Figure 7, bottom right). This starves the Conjecturer of any training signal, as problems with 0 or 1 solve rate get 0 reward. We observe different behavior when using REINFORCE$^{1/2}$ as the Solver objective. The Conjecturer is able to learn to produce problems with a spread of solve rates (Figure 7, bottom right). The presence of synthetic problems from the Conjecturer to train the Solver on appears to stabilize the Solver's entropy, which in turn provides the Conjecturer with an easier objective to learn. Overall, we find that the entropy of the Solver remains stable when using REINFORCE$^{1/2}$. We hypothesize that this connection between the Solver's entropy and Conjecturer's learning objective is key to making successful LLM self-play algorithms. It does not preclude the use of grouped objectives like CISPO for the Solver, but additional methods to manage the entropy collapse (such as the use of an entropy bonus or KL regularization to the base model) are required.

## 5 Limitations and Conclusion

**Extension to domains beyond formal math**. Our application of SGS to the verifiable domain of formal math makes the role of the Conjecturer easier as it only has to produce an initial state (problem statements) as opposed to an entire MDP (including reward function). We are excited about work extending SGS to non-verifiable domains. Consider, for example, applying SGS to embodied control. The Conjecturer would need to specify a goal, a simulator or environment in which the Solver can attempt that goal, and a reward function. One could imagine the goal being generated by an LLM in text, environment being generated by a neural world model, like Google DeepMind's Genie (Bruce et al., 2024), and a simple VLM acting as the reward function. For coding problems, the Conjecturer could output a problem statement and unit tests. For natural language mathematics, learned verifiers have improved rapidly (Luong et al., 2025) and could be used as-is to replace the formal verifier we use. More broadly, humans routinely learn from self-generated problems using only their own (imperfect) ability to verify solutions, suggesting that perfect verification may not be strictly necessary for self-play to be effective.

**Learning the Guide**. In our experiments, the Guide is frozen. While this is sufficient to prevent the Conjecturer collapse we observe in §4.4, it is likely insufficient for scaling SGS to solve the most challenging problems. In such cases, the Guide must itself learn what constitutes a useful synthetic problem, since the characteristics of helpful stepping stones will evolve over self-play. A natural extension is to train the Guide on aspects of the Solver training dynamics, for example labeling synthetic problems that quickly lead to their corresponding target being solved as high reward.

**Scaling across model size**. Our experiments focus on scaling along the compute axis but hold model size fixed. This leaves open the question of how SGS interacts with the model scale axis. We, naturally, hypothesize that the method will scale well in model size primarily because a larger Conjecturer will produce higher-quality synthetic problems. Confirming this hypothesis is an important direction for future work.

### 5.1 Conclusion

Asymmetric self-play is a promising paradigm for solving hard problems far beyond a base model's capability. To solve the hardest problems, self-play must run for a long time, making it critical that learning scales well with compute. Current methods scale poorly due to instability: the Conjecturer produces degenerate problems and the Solver can undergo entropy collapse, starving the Conjecturer of reward. SGS solves this: a Guide keeps synthetic problems grounded, and the right Solver objective preserves Solver entropy. The result is a self-play algorithm that sustains learning for longer.

## Acknowledgements

LB thanks the support of a Stanford Graduate and Vitalik Buterin Fellowship. TM thanks the support of NSF 2522743. KW thanks the support of the Stanford Graduate Fellowship. TH was supported by a grant by HAI, DSO labs, gifts from Open Philanthropy, Amazon, Schmidt Sciences, the Tianqiao and Chrissy Chen Foundation and a grant under the NSF CAREER IIS-2338866, ONR N00014-24-1-2609, and DARPA Cooperative Agreement HR00112520013. This work does not necessarily reflect the position or policy of the government and no official endorsement should be inferred. We thank Google TPU Research Cloud for the computing resources and Thinking Machines for providing Tinker compute credits that were used in testing for various parts of this project.

We thank Lars Ankile and Tanishq Kumar for helpful feedback throughout the project. We thank Tristan Thrush and Herman Brunborg for help testing our training infrastructure. We thank Neil Band, Caroline Choi, Thomas Chen, and Arvind Mahankali for feedback on an early draft of this work. We thank Mert Yuksekgonul and Yu Sun for useful conversations at the inception of the project, and writing feedback. We thank Noah Jun and Marka Ellertson for help with manuscript writing.

## References


Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.

Justin Yang Chae, Md Tanvirul Alam, and Nidhi Rastogi. Towards understanding self-play for llm reasoning. *arXiv preprint arXiv:2510.27072*, 2025.

Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025a.

Jiangjie Chen, Wenxiang Chen, Jiacheng Du, Jinyi Hu, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Wenlei Shi, et al. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience. *arXiv preprint arXiv:2512.17260*, 2025b.

Lili Chen, Mihir Prabhudesai, Katerina Fragkiadaki, Hao Liu, and Deepak Pathak. Self-questioning language models. *arXiv preprint arXiv:2508.03682*, 2025c.

Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, et al. Seed-prover: Deep and broad reasoning for automated theorem proving. *arXiv preprint arXiv:2507.23726*, 2025d.

Caroline Choi, Zeyneb N Kaya, Shirley Wu, Tengyu Ma, Tatsunori Hashimoto, and Ludwig Schmidt. Anchored self-play for code repair.

Leonardo De Moura, Soonho Kong, Jeremy Avigad, Floris Van Doorn, and Jakob von Raumer. The lean theorem prover (system description). In *International Conference on Automated Deduction*, pp. 378–388. Springer, 2015.

Kefan Dong and Tengyu Ma. Stp: Self-play llm theorem provers with iterative conjecturing and proving. *arXiv preprint arXiv:2502.00212*, 2025.

Carlos Florensa, David Held, Xinyang Geng, and Pieter Abbeel. Automatic goal generation for reinforcement learning agents. In *International conference on machine learning*, pp. 1515–1528. PMLR, 2018.

Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-zero: Self-evolving reasoning llm from zero data. *arXiv preprint arXiv:2508.05004*, 2025.

Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, pp. 1–3, 2025.

Swadesh Jana, Cansu Sancaktar, Tomáš Daniš, Georg Martius, Antonio Orvieto, and Pavel Kolev. Gasp: Guided asymmetric self-play for coding llms. *arXiv preprint arXiv:2603.15957*, 2026.

Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S Dhillon, David Brandfonbrener, and Rishabh Agarwal. The art of scaling reinforcement learning compute for llms. *arXiv preprint arXiv:2510.13786*, 2025.

Jakub Grudzien Kuba, Mengting Gu, Qi Ma, Yuandong Tian, Vijai Mohan, and Jason Chen. Language self-play for data-free training. *arXiv preprint arXiv:2509.07414*, 2025.

Gengsheng Li, Jinghan He, Shijie Wang, Dan Zhang, Ruiqi Liu, Renrui Zhang, Zijun Yao, Junfeng Fang, Haiyun Guo, and Jinqiao Wang. R-diverse: Mitigating diversity illusion in self-play llm training. *arXiv preprint arXiv:2602.13103*, 2026.

Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13(9):9, 2024.

Margaret Li, Sneha Kudugunta, and Luke Zettlemoyer. (mis) fitting: A survey of scaling laws. *arXiv preprint arXiv:2502.18969*, 2025.

Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, et al. Goedel-prover: A frontier model for open-source automated theorem proving. *arXiv preprint arXiv:2502.07640*, 2025a.

Yong Lin, Shange Tang, Bohan Lyu, Ziran Yang, Jui-Hui Chung, Haoyu Zhao, Lai Jiang, Yihan Geng, Jiawei Ge, Jingruo Sun, et al. Goedel-prover-v2: Scaling formal theorem proving with scaffolded data synthesis and self-correction. *arXiv preprint arXiv:2508.03613*, 2025b.

Bo Liu, Chuanyang Jin, Seungone Kim, Weizhe Yuan, Wenting Zhao, Ilia Kulikov, Xian Li, Sainbayar Sukhbaatar, Jack Lanchantin, and Jason Weston. Spice: Self-play in corpus environments improves reasoning. *arXiv preprint arXiv:2510.24684*, 2025.

Minh-Thang Luong, Dawsen Hwang, Hoang H Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, et al. Towards robust mathematical reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 35406–35430, 2025.

Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pp. 625–635. Springer, 2021.

Gabriel Poesia, David Broman, Nick Haber, and Noah Goodman. Learning formal mathematics from intrinsic motivation. *Advances in Neural Information Processing Systems*, 37:43032–43057, 2024.

Jatin Prakash and Anirudh Buvanesh. What can you do when you have zero rewards during rl? *arXiv preprint arXiv:2510.03971*, 2025.

Yuxiao Qu, Amrith Setlur, Virginia Smith, Ruslan Salakhutdinov, and Aviral Kumar. Pope: Learning to reason on hard problems via privileged on-policy exploration. *arXiv preprint arXiv:2601.18779*, 2026.

ZZ Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, et al. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.

Yangjun Ruan, Chris J Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of langauge model performance. *Advances in Neural Information Processing Systems*, 37:15841–15892, 2024.

Arthur L Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of research and development*, 3(3):210–229, 1959.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

Sainbayar Sukhbaatar, Zeming Lin, Ilya Kostrikov, Gabriel Synnaeve, Arthur Szlam, and Rob Fergus. Intrinsic motivation and automatic curricula via asymmetric self-play. *arXiv preprint arXiv:1703.05407*, 2017.

Shobhita Sundaram, John Quan, Ariel Kwiatkowski, Kartik Ahuja, Yann Ollivier, and Julia Kempe. Teaching models to teach themselves: Reasoning at the edge of learnability. *arXiv preprint arXiv:2601.18778*, 2026.

Gerald Tesauro et al. Temporal difference learning and td-gammon. *Communications of the ACM*, 38(3):58–68, 1995.

Sebastian Thrun. Learning to play the game of chess. *Advances in neural information processing systems*, 7, 1994.

Sumanth Varambally, Thomas Voice, Yanchao Sun, Zhifeng Chen, Rose Yu, and Ke Ye. Hilbert: Recursively building formal proofs with informal reasoning. *arXiv preprint arXiv:2509.22819*, 2025.

Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, Junqi Liu, Marco Dos Santos, Flood Sung, Marina Vinyes, Zhenzhe Ying, Zekai Zhu, et al. Kimina-prover preview: Towards large formal reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.11354*, 2025.

Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint arXiv:2511.16043*, 2025.

Chengyi Yang, Zhishang Xiang, Yunbo Tang, Zongpei Teng, Chengsong Huang, Fei Long, Yuhan Liu, and Jinsong Su. Ttcs: Test-time curriculum synthesis for self-evolving. *arXiv preprint arXiv:2601.22628*, 2026.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale, 2025. *URL https://arxiv. org/abs/2503.14476*, 1:2, 2025a.

Wenhao Yu, Zhenwen Liang, Chengsong Huang, Kishan Panaganti, Tianqing Fang, Haitao Mi, and Dong Yu. Guided self-evolving llms with minimal human supervision. *arXiv preprint arXiv:2512.02472*, 2025b.

Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, M Lin, S Wang, Q Wu, Z Zheng, and G Huang. Absolute zero: Reinforced self-play reasoning with zero data, 2025. *URL https://arxiv. org/abs/2505.03335*.

# A Fitting Cumulative Solve Rate

Following other works modeling accuracy metrics (Khatri et al., 2025; Ruan et al., 2024), we adopt a sigmoidal curve (with respect to log compute) of the form

$$R_C = R_0 + (A - R_0) \cdot \frac{1}{1 + (C_{\text{mid}}/C)^B}, \tag{2}$$

where $R_C$ is the cumulative solve rate at $C$ number of model generations (our proxy for compute), $R_0$ is the initial solve rate, $A$ is the asymptotic solve rate, $C_{\text{mid}}$ sets the midpoint of the curve, and $B$ controls the rate at which performance approaches the asymptote.

We fit the parameters $A, C_{\text{mid}}, B$ by minimizing the sum of squared residuals using SciPy's `curve_fit`. The initial performance $R_0$ is set to the first observed solve rate rather than treated as a free parameter.

Consistent with observations in both pre-training scaling analyses (Li et al., 2025) and RL scaling studies (Khatri et al., 2025) we find that the earliest data points, corresponding to the low-compute regime, can disproportionately influence the fitted parameters. In our setting, the first few iterations exhibit rapid gains that are not representative of the longer-term scaling trend. We therefore omit all data below 100,000 generations from the fit and re-center the remaining data, effectively modeling the cumulative gains after the initial low-compute phase. Empirically we found this increased the robustness of our fits.

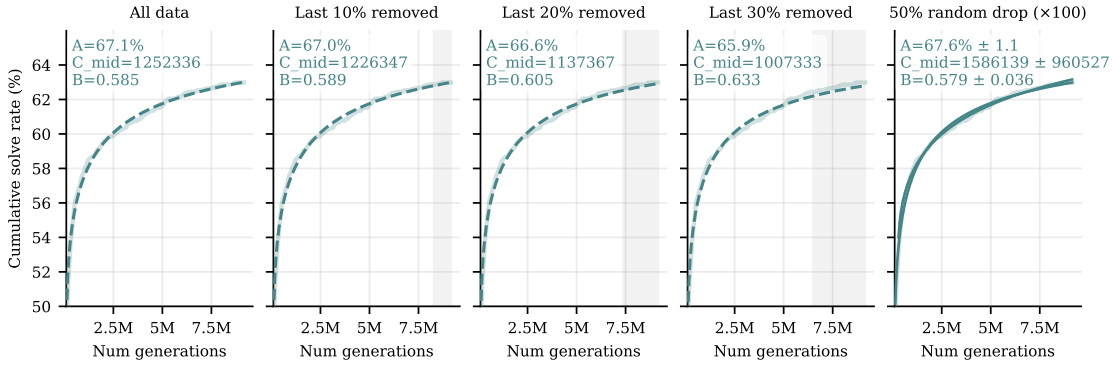


Figure 8: Sensitivity of the fitted scaling law to data removal for our main SGS run. **Panels 1–4:** Fits with various amounts of training data removed from the end of the run. The fitted values stay fairly robust, with a 1.6% decrease in asymptote value after 30% of the ending data is removed. **Panel 5:** 100 independent fits, each using a uniformly sampled 50% subset of the data (without replacement). The average and standard deviation of fitted values is reported on the graph, as well as the fit with the highest and lowest asymptotic value across all 100 fits. In this test, the fitted values are also stable, with a 1.1% standard deviation in the asymptotic value.

**Validation of fits**. We validate the robustness of our fits in two ways. First, we find that fits remain stable after removing 10%, 20%, and 30% of the ending training data. Second we find the fits are robust to randomly dropping 50% of the training data, with the standard deviation of fitted asymptote values being only 1.1% (see Figure 8 for more details). This does however indicate that

we should be cautious about drawing conclusions from asymptotic fits that are within 1.1% of one another. Of course, we cannot know the true asymptotic solve rate of the algorithms we test. We find the fitted scaling laws are a useful *additional* metric to compare the performance of different algorithms, but not a substitute for real long-running performance, hence our focus on this kind of real data collection as well as curve fitting.

# B Training Infrastructure

Each iteration of SGS alternates between a *rollout stage* (generation and verification) and a *training stage*. We coordinate two services during the rollout stage: a GPU-backed generation service and a CPU-backed verification service that checks proofs with the Lean4 compiler.

Both services follow a server–worker architecture: a central HTTP server dispatches tasks in small batches to stateless workers launched as independent Slurm jobs. When a worker requests a task, the server assigns one from the pending queue. Completed results are submitted back via HTTP POST. Generation and verification are fully pipelined: completed proofs are forwarded for verification immediately, so GPUs and CPUs stay saturated concurrently.

We implement two mechanisms to handle worker failures and stragglers. First, automatic worker restart: when a worker reports itself as dead (or times out), the server requeues its assigned tasks and launches a replacement worker if work remains. Second, speculative reassignment: when no pending tasks remain but some are still in progress, the server assigns duplicate copies of in-progress tasks to idle workers, preferring the task with the fewest current workers. The first worker to return a result wins; the task is removed from all other workers.

For generation, we use up to 64 GPU workers across multiple Slurm partitions, each with 1 GPU and 32 GB memory. For verification, we use up to 128 CPU workers, each with 33 CPU threads and 100 verification tasks per job. Training uses a single H200 node with ZeRO Stage-2 distributed optimization. All models use bfloat16 precision and a maximum sequence length of 8192 tokens.

For verification, we use Lean version 4.15.0, timeout set to 200s, and no memory limit. We monitor the rate of verification system errors throughout all runs, and ensure this never exceeds 1% of total verifications.

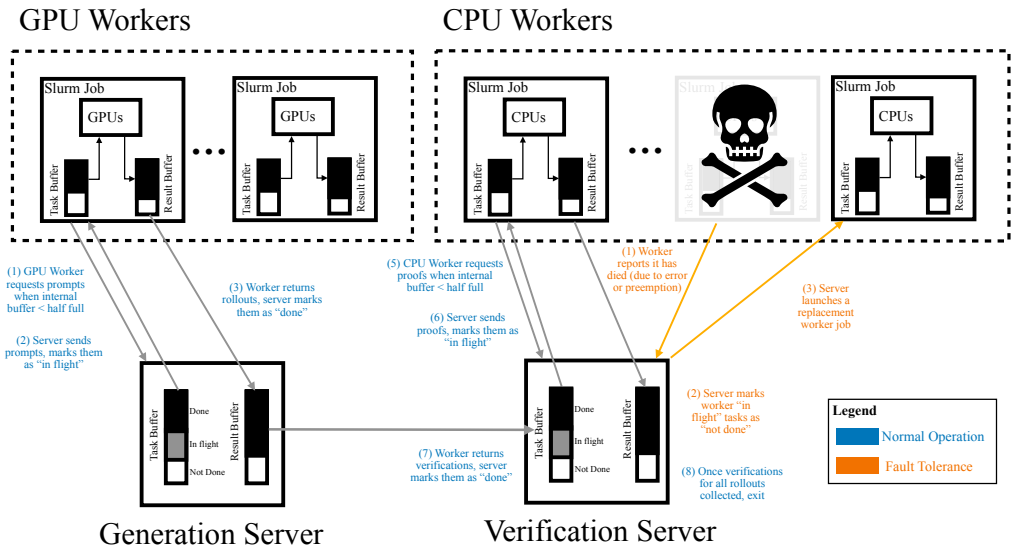


Figure 9: Diagram of inference infrastructure used for experiments. We show the normal operation of the infrastructure (**blue**), and the fault tolerance mechanisms (**orange**). Note, the diagram shows the fault tolerance mechanisms for the Verification service, but it is used identically for the Generation service.

# C Hyperparameters

Here we include details of the hyperparameters used in our experiments, and the selection mechanisms.

## C.1 Shared Hyperparameters

All methods use the Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.95$, gradient clipping at norm 1.0, and a constant learning rate schedule. We use a batch size of 32 (per-GPU batch size of 1, with ZeRO Stage-2 data parallelism). The maximum sequence length is 8192 tokens. All generation uses a temperature of 1.0. For CISPO, we set $\epsilon_{\text{low}} = 1.0$ and $\epsilon_{\text{high}} = 3.0$ (clipping the importance weight to $[0, 4]$).

## C.2 Learning Rate Selection

To ensure the strongest possible baselines, we sweep over learning rates {3e-7, 1e-6, 3e-6, 1e-5} for all three RL baselines on $D_{3\text{k}}$, selecting the learning rate with the highest cumulative solve rate after 500k generations. In all cases we found the best performing learning rate was not boundary point of these values (we originally did not test 1e-5, however expanded to include this value when 3e-6 appeared to be optimal for REINFORCE$^{1/2}$). The optimal learning rate for CISPO is 1e-6, while both EI and REINFORCE$^{1/2}$ perform best at 3e-6. For SGS, we inherit the 3e-6 learning rate from the REINFORCE$^{1/2}$ sweep, using it for both the Solver and Conjecturer.

## C.3 Individual Experiment Hyperparameters

All experiments use the shared hyperparameters from the preceding subsection and the learning rates described in the learning rate selection subsection. We provide a summary of per-experiment configurations in Table 1.

| Experiment | Figure(s) | LR | Solver Obj. | Synth. | Guide |
|---|---|---|---|---|---|
| SGS | 1, 6, 7 | 3e-6 | REINFORCE$^{1/2}$ | 1 | ✓ |
| REINFORCE$^{1/2}$ | 1, 4, 6, 7 | 3e-6 | REINFORCE$^{1/2}$ | 0 | — |
| Expert Iteration | 4 | 3e-6 | EI | 0 | — |
| CISPO | 4, 7 | 1e-6 | CISPO | 0 | — |
| No Guide | 6 | 3e-6 | REINFORCE$^{1/2}$ | 1 | ✗ |
| Frozen Conjecturer | 6 | 3e-6 | REINFORCE$^{1/2}$ | 1 | ✗ |
| No Problem Cond. | 6 | 3e-6 | REINFORCE$^{1/2}$ | 1 | ✗ |
| CISPO Solver | 7 | 1e-6 | CISPO | 1 | ✓ |

Table 1: Per-experiment hyperparameters. **Synth.**: number of synthetic problems generated per unsolved target statement. All other hyperparameters (optimizer, batch size, gradient clipping, sequence length, CISPO clipping bounds) are as described in the shared hyperparameters subsection.

# D Data Details

We detail the method we use to create the $D_{3k}$ dataset. We begin with Goedel-Pset-V1 (Lin et al., 2025a), a large collection of auto formalized problems primarily from the NuminaMath-CoT dataset (Li et al., 2024). We conduct the following filtering steps:

1. **First pass removing easy problems.** Remove any problem with a pass rate of above 50% (estimated from 8 rollouts per proof) using the STP 7B lean prover model from Dong & Ma (2025).
2. **First pass removing false problems, second pass removing easy problems.** Next we take a slightly more capable prover, Kimina-Prover-Preview (trained with a large-scale reinforcement learning pipeline from Qwen2.5-72B (Wang et al., 2025)) and do 8 rollouts for every problem and for *the negation of every problem*. If it is possible to prove the negation of the problem, then the original statement was false (and thus unprovable). We remove any problem where the negation was proved at least once, and as a second pass removing easy problems, we remove any problems that Kimina-Prover-Preview proves more than 2/8 times.
3. **Subsampling.** As we are interested in studying a regime with many rounds of self-play, we subsample the dataset down to 5,000 problems randomly.
4. **Final pass removing false problems.** Finally, we take the remaining 5,000 problems, and prompt GPT 5 mini to identify any problems that it thinks are false. We prompt the model to reason and then output one of 'solvable', 'unknown', 'unsolvable'. We remove any problems given the label 'unsolvable'. This leaves us with 3,323 problems, which make up $D_{3k}$. The full prompt is provided in §E.4.

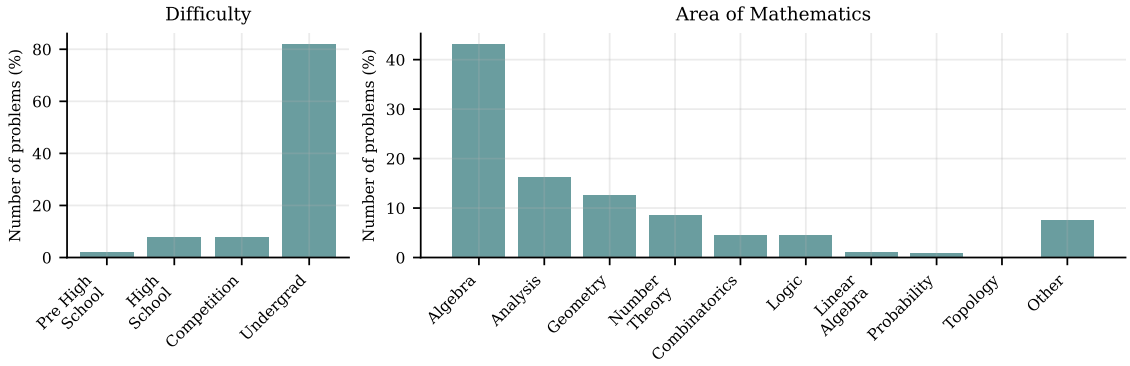


Figure 10: Distribution of problem types and difficulty in $D_{3k}$.

## D.1 Composition of $D_{3k}$

We release $D_{3k}$ publicly. [1] We provide details about the dataset composition in Figure 10. Specifically, for each problem in $D_{3k}$ we prompt GPT 5.4 nano to categorize the difficulty of the problem as

[1] $D_{3k}$ can be found at https://huggingface.co/datasets/LukeBailey181Pub/D_3k

one of {Pre High School, High School, Competition, Undergraduate} and topic as one of {Algebra, Analysis, Geometry, Number Theory, Combinatorics, Logic, Linear Algebra, Probability, Topology, Other}. For any problem that GPT 5.4 nano provides an output that we cannot parse, we reprompt until we have categorizations for over 99% of the dataset. We provide details of the prompt used for this categorization in §E.5.

# E Prompting Details

We describe the prompts used for the Conjecturer and Guide roles in SGS when using DeepSeek-Prover-V2-7B as the base model.

## E.1 Conjecturer Prompt

The Conjecturer is prompted with an unsolved target problem and asked to produce a related lemma or theorem. The full prompt is as follows:

```
Here is a Lean 4 problem statement:

```lean4
{target_theorem}
```

Please generate a lean4 theorem that is a lemma or related theorem that
is useful for proving the above statement.  It should be possible to use
the lemma or related theorem to help prove the above statement.  The lemma
or related theorem should be simpler to prove than the target statement.
It should NOT be identical to the target statement.  It should NOT be equivalent
through renaming variables and premises.

Output the final theorem as a syntactically correct lean4 theorem statement
between ```lean4 and ``` tags.  The final thing you output should be the
theorem statement WITHOUT any proof, just put 'sorry' for the proof.
```

## E.2 No Problem Conditioning Prompt

In the No Problem Conditioning ablation (§4.4), the Conjecturer is not conditioned on any target problem. Instead, it is prompted to generate an arbitrary theorem:

```
Please generate a Lean4 theorem statement.  The theorem should be non-trivial,
but not too difficult to prove.  Choose a random area of mathematics for
the theorem, such as algebra, number theory, combinatorics, geometry, calculus,
or logic.  The theorem should be interesting and mathematically meaningful,
not just a very easy identity or direct restatement of a definition.

Output the final theorem as a syntactically correct Lean4 theorem statement
between ```lean4 and ``` tags.  The final thing you output should be the
theorem statement WITHOUT any proof, and just put 'sorry' for the proof.
```

## E.3 Guide Prompt

The Guide is a finetuned copy of DeepSeek-Prover-V2-7B that scores each synthetic problem on three dimensions. We finetune on 2048 examples of well-formatted outputs generated by GPT-4.1 mini to ensure reliable output formatting (see §4.1). Given a target theorem and a generated conjecture, the Guide is prompted as follows:

You are a math expert. Here is a seed lean4 problem statement:

```lean4
{seed_theorem}
```

Here is a lemma or related lemma that is supposed to be useful for proving the above statement. It can be useful in that either it is a lemma that can be directly used to help prove the above statement, or it is a related lemma that plausibly requires similar proof techniques to solve.

```lean4
{conjecture}
```

Please rate the relevance of the lemma to the seed problem on a scale of 0 to 5, where 0 is ``not at all related'' and 5 is ``very useful for proving the target statement.'' If the lemma is trivial, you should give it a low score.

Here is a rubric for how to score the lemma:
- 0: The lemma is not at all related to the seed problem and is trivial to prove. OR the lemma is identical (including equivalent by renaming of premises and variables) to the seed problem.
- 1: The lemma is not at all related to the seed problem.
- 2: The lemma is related to the seed problem in that it concerns a similar subfield of mathematics, but is not directly useful for proving the seed problem.
- 3: The lemma is related to the seed problem and may be useful for proving the seed problem.
- 4: The lemma is directly useful for solving the seed problem. That is if the lemma was proved, the seed problem would be easier to solve.
- 5: The lemma is very useful for solving the seed problem, and solving the lemma will dramatically reduce the difficulty of the original seed problem.

Next decide how redundant the premises are. Rate this as 0 or 1:
- 0: There are no redundant premises.
- 1: There are redundant premises. That is premises that are not needed to prove the conclusion.

Next decide if the conclusion is overly complex. Rate this on a score of 0 to 4:
- 0: The conclusion is minimally complex. A single, atomic statement that is maximally clear and easy to apply to other problems.
- 1: The conclusion has low complexity. Multiple related parts (e.g., 2--3 conjunctions) but they form a cohesive statement.
- 2: The conclusion has moderate complexity. Contains disjunctions but they are closely related alternatives, or contains multiple conjunctions (3--4) that are all on-theme.
- 3: The conclusion has high complexity. Contains multiple unrelated or clauses (2--3 disjunctions), or contains deep nesting of logical operators that obscure the main claim.

```
- 4:  The conclusion has very high complexity.  A disjunction of many (3
or more) largely unrelated clauses, or contains deeply nested logical structure
that is hard to parse.
```

The three sub-scores are combined into a single review score as follows. If the conclusion complexity is 3 or 4 (high or very high), the review score is automatically 0. Otherwise:

$$R_{\text{guide}} = \max\Big(0,\ \text{relevance} + (2 - \text{complexity}) + (1 - \text{redundancy})\Big). \tag{3}$$

This rewards relevant conjectures with simple, non-redundant conclusions, and assigns zero reward to conjectures with overly complex conclusions.

## E.4 Solvability Filtering Prompt

During dataset construction (§D), we use GPT 5 mini to filter out unsolvable problems. Given a candidate problem, the model is prompted as follows:

```
Here is a lean4 problem:

```lean4
{theorem}
```

I want you to determine if this problem is solvable.  There are three possible
answers:

1) Yes, the problem is solvable.  That is there exists some proof of the
problem.  Note that if two premises contradict each other, then the problem
IS solvable as it is vacuously true.
2) Unknown.  You are not sure if the problem is solvable or not.
3) No, the problem is not solvable.  I.e. the conclusion is not provable
from the premises.

Reason about the problem and then decide on an answer from the above.  If
your answer is 1), output <SOLVABLE>.  If your answer is 2), output <UNKNOWN>.
If your answer is 3), output <UNSOLVABLE>.

Make sure to only output one of these three tags.
```

Problems receiving the `<UNSOLVABLE>` label are removed from the dataset.

## E.5 Dataset Categorization Prompt

To categorize the difficulty and topic of each problem in $D_{3k}$ (§D), we use GPT 5.4 nano with the following prompt:

```
Classify this Lean 4 theorem.  Respond with ONLY two words:  difficulty
area.  No reasoning.
```

```
Difficulty:  pre_high_school, high_school, competition, undergraduate
Area:  algebra, analysis, number_theory, combinatorics, geometry, topology,
probability, linear_algebra, logic, other

Example response:  undergraduate algebra

Theorem:
{theorem}
```

Any problems we cannot parse the answer from, we reprompt until we have over 99% of the dataset classified.

# F Additional Results

In this section we provide additional results from the experiments conducted in §4.

## F.1 Baseline Entropy

Figure 11 is the companion figure to Figure 4, showing the entropy of the Solver's output distribution for each baseline method. We see that CISPO suffers the fastest entropy collapse, REINFORCE$^{1/2}$ entropy also decreases but at a slower rate, and EI entropy plateaus as late in training the number of problems it trains on becomes extremely small.

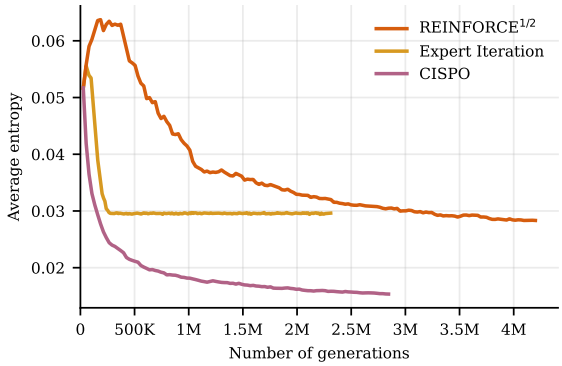


Figure 11: Mean per-token entropy of the Solver's output distribution for each RL baseline on $D_{3k}$. CISPO collapses fastest, followed by Expert Iteration which plateaus at a low entropy floor early in training. Note the entropy values are naturally lower than one would expect from LLMs as most of the rollouts are Lean specific tokens.

## F.2 Comparison to STP

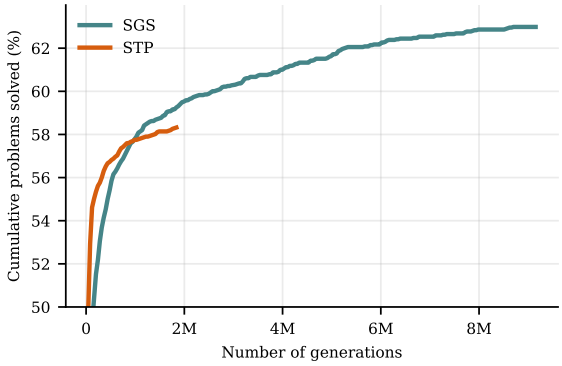


Figure 12: SGS and STP (Dong & Ma, 2025) performance on $D_{3k}$. We see that SGS has superior scaling, with a crossover happening at around 1M generations.

We compare SGS to STP (Dong & Ma, 2025), the closest related self-play method designed *specifically for Lean4 theorem proving*. To run STP we take the code released by (Dong & Ma, 2025). This code is designed to run on TPUs, so we use 64 v4 TPUs as opposed to GPUs. To get better hardware utilization, we also increase the batch size to 64. For hyperparameter optimization, we go through a similar process to SGS. In particular, the Solver algorithm used in STP is EI, and thus we take the optimal EI learning rate we found of 3e-6, and keep the rest of the hyperparameters the same as described in §C.1. The results are shown in Figure 12. We see that SGS has superior scaling, with a crossover happening at around 1M generations.

### F.3 Performance on problems beyond RL Baseline

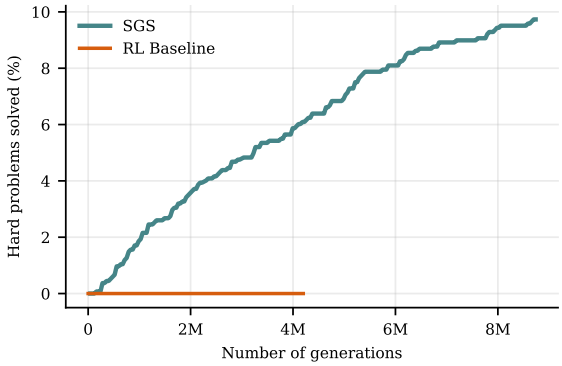


Figure 13: Performance on $D_{\text{hard}}$, the subset of $D_{3\text{k}}$ never solved by REINFORCE$^{1/2}$, during the main run in Figure 1. SGS steadily solves a growing fraction of these problems.

Taking the RL baseline (REINFORCE$^{1/2}$) and SGS run data from Fig. 1, we derive a new dataset $D_{\text{hard}} \subset D_{3\text{k}}$ of all the problems that the RL baseline never solved. We get $|D_{\text{hard}}| = 1346$, roughly $1/3$ the size of $D_{3\text{k}}$. In Figure 13 we show the percent of problems in $D_{\text{hard}}$ that are solved by SGS and the RL baseline during the main experimental run (Fig. 1). By construction, the RL baseline solves 0 problems. However, SGS shows steady progress on this set of hard problems that are inaccessible to RL, reaching close to a 10% cumulative solve rate.

$D_{\text{hard}}$ isolates the difficult problems that (within 4M generations) our best RL method gets *0 reward on*. Despite this, we see SGS making steady and non-trivial progress on such problems from the very beginning of the run.

# G RL Objective Functions

We describe the RL objectives used in our experiments. At each iteration, we sample a batch of problems $\mathcal{B} \subseteq \mathcal{D}$ and generate $k$ rollouts $\{y^i\}_{i=1}^k$ per problem $x \in \mathcal{B}$. Each rollout is verified by the Lean4 compiler, yielding a binary reward $v(y) \in \{0, 1\}$.

## G.1 REINFORCE

The base REINFORCE objective is a log-likelihood update weighted by the binary reward:

$$\mathcal{L}_{\text{RF}}(\theta) = -\,\mathbb{E}_{x\sim\mathcal{B},\, y\sim\pi_{\theta_{\text{old}}}(\cdot|x)} \left[ \frac{1}{|y|} \sum_{t=1}^{|y|} v(y)\, \log \pi_\theta\big(y_t \mid x, y_{<t}\big) \right] \tag{4}$$

## G.2 REINFORCE$^{1/2}$

REINFORCE$^{1/2}$ is our primary Solver objective. It applies the REINFORCE loss above, but only to problems with solve rate $s(x) = \frac{1}{k}\sum_{i=1}^k v(y_x^i) \leq 0.5$. All problems with $s(x) > 0.5$ are discarded from the training batch. This prevents the Solver from spending gradient updates on already-easy problems, focusing learning on problems where progress is still needed.

## G.3 CISPO

CISPO is a grouped reinforcement learning objective introduced by Chen et al. (2025a). The recent study by Khatri et al. (2025) found it to have the best asymptotic performance of LLM RL algorithms developed in the past year. Like GRPO (Shao et al., 2024), CISPO uses the group of rollouts $\{y^i\}_{i=1}^k$ from a single prompt $x$ to calculate an advantage, however it uses an improved importance sampling clipping method compared to the widely used PPO-style clipping (Schulman et al., 2017).

For a problem $x$ and its rollout group $\{y^i\}_{i=1}^k$, where each rollout $y^i = (y_{i,1}, \ldots, y_{i,|y^i|})$, let

$$w_{i,t} = \frac{\pi_\theta\big(y_{i,t} \mid x, y_{i,<t}\big)}{\pi_{\theta_{\text{old}}}\big(y_{i,t} \mid x, y_{i,<t}\big)}$$

be the token-level importance sampling weight. Let

$$\hat{A}^i = \frac{v(y^i) - \text{mean}(\{v(y^j)\}_{j=1}^k)}{\text{std}(\{v(y^j)\}_{j=1}^k)} \quad \text{and}$$

$$\hat{w}_{i,t} = \text{clip}\big(w_{i,t},\ 1-\epsilon_{\text{low}},\ 1+\epsilon_{\text{high}}\big)$$

be the group-relative advantage and clipped token-level importance sampling weight respectively, where $\epsilon_{\text{low}}$ and $\epsilon_{\text{high}}$ are hyperparameters. The CISPO loss is:

$$\mathcal{L}_{\text{CISPO}}(\theta) = -\mathbb{E}_{x\sim\mathcal{B},\, \{y^i\}_{i=1}^{k}\sim\pi_{\theta_{\text{old}}}(\cdot|x)}$$
$$\left[\frac{1}{\sum_{i=1}^{k}|y^i|}\sum_{i=1}^{k}\sum_{t=1}^{|y^i|}\text{sg}(\hat{w}_{i,t})\,\hat{A}^i\,\log\pi_\theta\big(y_{i,t}\mid x, y_{i,<t}\big)\right] \tag{5}$$

where sg is the stop-gradient operator.

## G.4 Expert Iteration

Our Expert Iteration (EI) baseline, following Dong & Ma (2025), only generates rollouts for problems that have been solved fewer than 16 times in prior iterations. Training is performed on correct proofs from the last 3 iterations using the REINFORCE objective. This focuses compute on problems where the model has limited coverage, but we find that late in training the number of newly solved problems per iteration tends to zero, causing learning to stall.

## G.5 Conjecturer Objective

The Conjecturer is trained with REINFORCE using the combined reward $R_{\text{synth}} = R_{\text{solve}} \cdot R_{\text{guide}}$, as described in §3. The rewards are linearly normalized to $[0, 1]$ within each batch before computing the policy gradient. We do so by replacing $r_i$ with $(r_i - \min_j r_j)/(\max_j r_j - \min_j r_j)$. Conjectures with a solve rate of 0 (unsolvable) or in the top 30% of solve rates within the batch (too easy) receive $R_{\text{solve}} = 0$.